Chest X-Ray Pneumonia Detection Using Deep Learning & Transfer Learning

A Complete End-to-End Deep Learning Project for Medical AI

PyTorch | EfficientNet-B0 | Transfer Learning | Grad-CAM | Google Colab

Framework	Base Model	Dataset Size	Classes	Target Accuracy	Explainability
PyTorch	EfficientNet-B0	5,856 Images	NORMAL / PNEUMONIA	92% — 96%	Grad-CAM

Abstract

This article presents a comprehensive, end-to-end deep learning pipeline for automated detection of pneumonia from chest X-ray images. The system leverages EfficientNet-B0, pretrained on ImageNet, with a two-phase Transfer Learning strategy: classifier head training followed by selective fine-tuning of deep convolutional blocks. The pipeline incorporates medical-grade data augmentation, manual early stopping, best-weight checkpointing, and Grad-CAM visual explainability — making it suitable for real-world clinical decision support. The complete implementation runs on Google Colab with GPU acceleration and is deployable via Gradio or Hugging Face Spaces.

1. Introduction

1.1 The Clinical Problem

Pneumonia is one of the deadliest infectious diseases worldwide. According to the World Health Organization (WHO), it claims the lives of over 2.5 million people annually and remains the leading cause of mortality in children under five years of age, particularly in low-resource settings.

Conventional diagnosis relies on radiologists manually examining chest X-rays — a process that is time-consuming, subject to inter-observer variability, and severely constrained by the global shortage of trained radiologists. In many developing regions, a single radiologist may be responsible for reviewing hundreds of images daily, inevitably increasing the risk of diagnostic error.

Project Goal

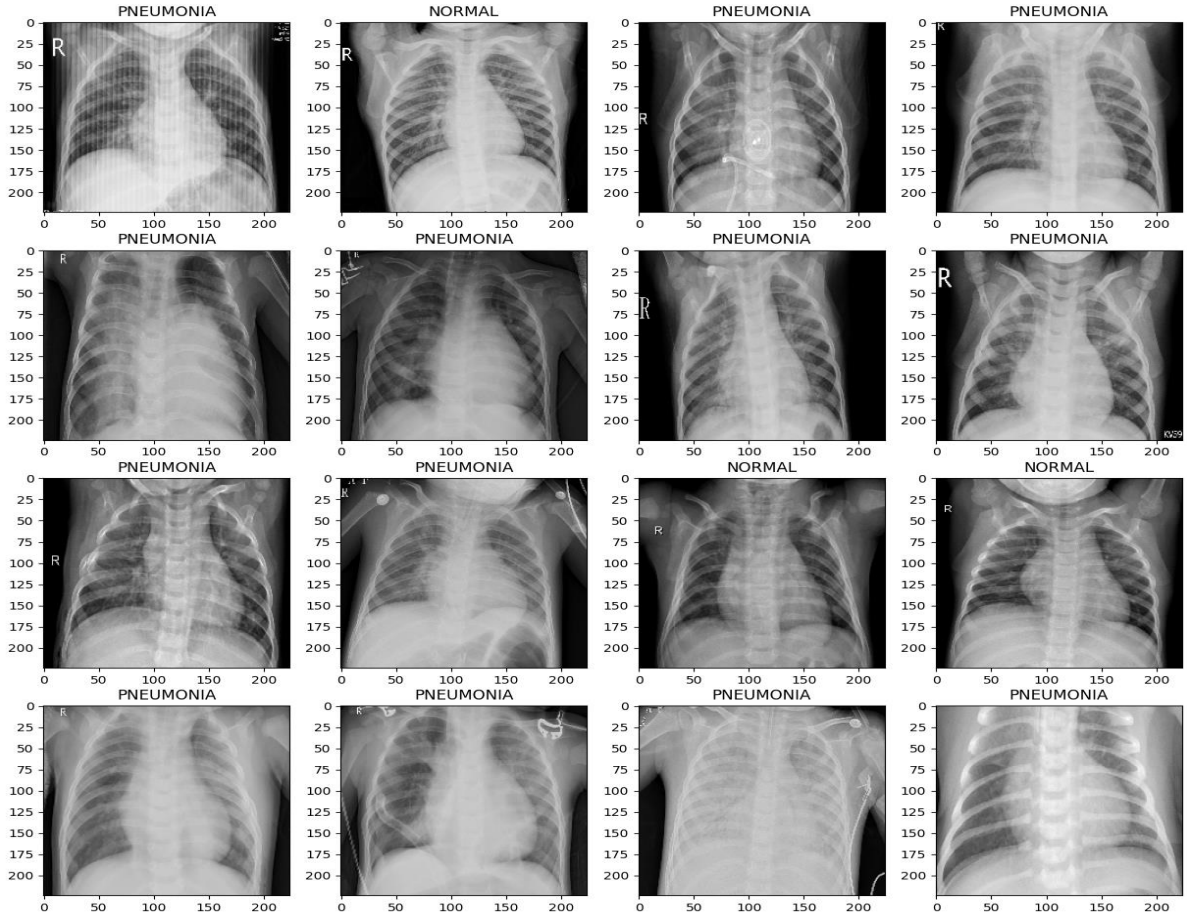
Build an AI-powered system capable of classifying chest X-ray images as NORMAL or PNEUMONIA with high accuracy and clinical interpretability — augmenting, not replacing, the radiologist's judgment.

1.2 Why This Matters

- Speed: The model classifies an image in under one second vs. minutes per image for a radiologist.
- Consistency: Unlike humans, the model does not suffer from fatigue or cognitive bias.
- Scalability: A trained model can be deployed globally at negligible marginal cost.
- Decision Support: Grad-CAM heatmaps provide visual justification that clinicians can verify or override.

2. Dataset

2.1 Data Description



Load data and install libraries

```
# 1. Install dependencies
!pip install grad-cam -q

# 2. Download the dataset (requires Kaggle API key)
!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia
!unzip chest-xray-pneumonia.zip -d .
import os
import copy
import time
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import seaborn as sns
from PIL import Image

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
)

# pytorch-grad-cam library (pip install grad-cam)
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

print("All libraries imported successfully.")
print(f"PyTorch version : {torch.__version__}")
```

The project uses the publicly available Chest X-Ray Images (Pneumonia) dataset from Kaggle, curated by Guangzhou Women and Children's Medical Center. It contains pediatric chest X-rays from patients aged 1 to 5 years, annotated by expert physicians and verified by a third-party grader.

Split	NORMAL	PNEUMONIA	Total	Notes
Train	1,341	3,875	5,216	Primary training data
Validation	8	8	16	Very small — prone to variance
Test	234	390	624	Final evaluation only
Total	1,583	4,273	5,856	—

Class Imbalance Alert

PNEUMONIA accounts for ~73% of training samples. This imbalance can bias the model toward over-predicting pneumonia. We address this through aggressive data augmentation rather than oversampling, preserving the natural distribution of the imaging modality.

5. Data Augmentation

Data augmentation is critical given the limited dataset size and class imbalance. All transformations are chosen to reflect realistic variation in real-world chest X-ray acquisition conditions.

Transformation	Parameters	Clinical Justification
RandomCrop	224×224 from 244×244	Simulates variability in centering during patient positioning
RandomHorizontalFlip	p = 0.5	Mimics anatomical left/right mirroring across patients
RandomRotation	±15 degrees	Accounts for slight patient rotation during imaging
ColorJitter (Brightness)	±0.3	Simulates variation in X-ray machine exposure settings
ColorJitter (Contrast)	±0.3	Mimics differences in detector sensitivity across devices
Normalize	ImageNet mean/std	Required for compatibility with pretrained EfficientNet weights

Important Note

RandomVerticalFlip is intentionally excluded. An upside-down chest X-ray is not a clinically realistic scenario and would introduce noise, not useful variance. Augmentation must always reflect plausible real-world variation.

```python
# Training transforms  — augmentation to prevent overfitting
train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE + 20, IMG_SIZE + 20)),  # slight oversize then crop
    transforms.RandomCrop(IMG_SIZE),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3),
    transforms.ToTensor(),
    # ImageNet mean/std — EfficientNet was pretrained on ImageNet
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std =[0.229, 0.224, 0.225]),
])

# Validation & Test transforms — no augmentation, just resize + normalize
val_test_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std =[0.229, 0.224, 0.225]),
])

print("Transforms defined.")
```

```
Transforms defined.
```

```python
train_dataset=datasets.ImageFolder(root=os.path.join(DATA_DIR,"train"),transform=train_transforms,)
test_dataset=datasets.ImageFolder(root=os.path.join(DATA_DIR,"test"),transform=val_test_transforms,)
val_dataset=datasets.ImageFolder(root=os.path.join(DATA_DIR,"val"),transform=val_test_transforms,)

train_loader =DataLoader(train_dataset,batch_size=BATCH_SIZE,shuffle=True,num_workers=2,pin_memory=True)
test_loader=DataLoader(test_dataset,batch_size=BATCH_SIZE,shuffle=False,num_workers=2,pin_memory=True)
val_loader=DataLoader(val_dataset,batch_size=BATCH_SIZE,shuffle=False,num_workers=2,pin_memory=True)
```

Show Samples of data

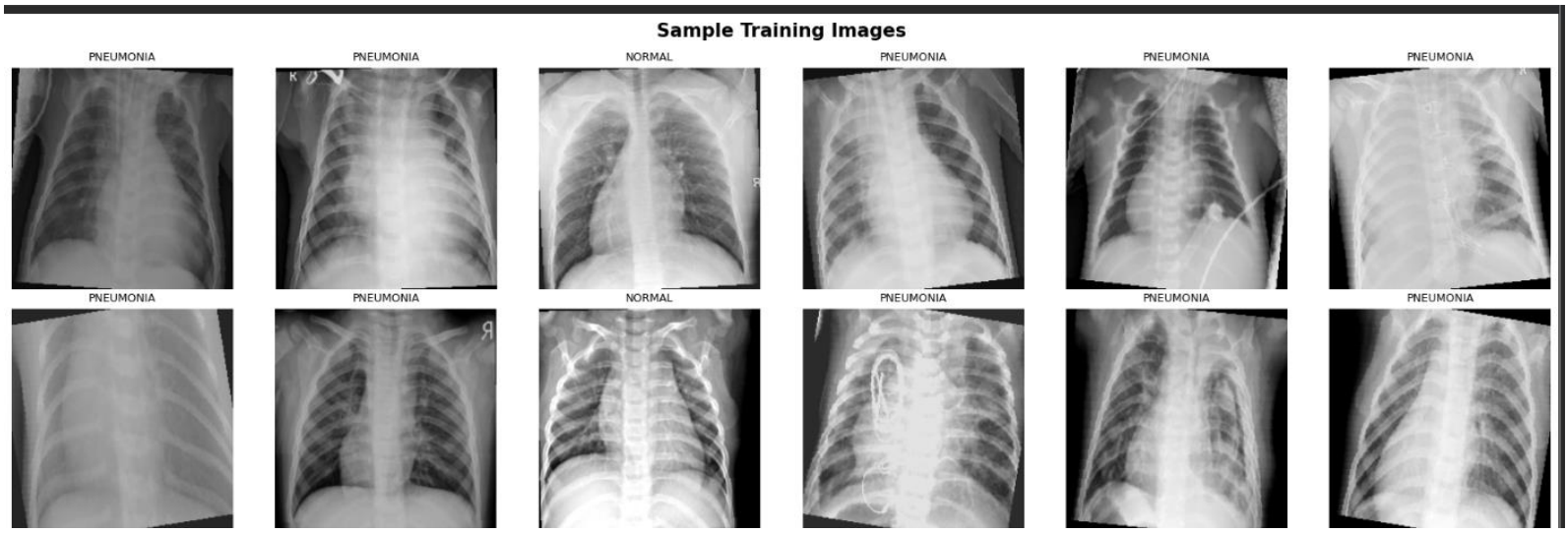
```python
# Un-normalise for display
INV_MEAN = torch.tensor([0.485, 0.456, 0.406])
INV_STD  = torch.tensor([0.229, 0.224, 0.225])

sample_images, sample_labels = next(iter(train_loader))

fig, axes = plt.subplots(2, 6, figsize=(18, 6))
fig.suptitle("Sample Training Images", fontsize=15, fontweight="bold")

for i, ax in enumerate(axes.flat):
    if i >= len(sample_images):
        break
    img = sample_images[i].cpu().clone()
    img = img * INV_STD[:, None, None] + INV_MEAN[:, None, None]
    img = img.permute(1, 2, 0).clamp(0, 1).numpy()
    ax.imshow(img)
    ax.set_title(Class_Names[sample_labels[i].item()], fontsize=9)
    ax.axis("off")

plt.tight_layout()
plt.savefig("sample_images.png", dpi=100, bbox_inches="tight")
plt.show()
print("Sample images saved.")
```



3. Model Architecture

3.1 Why EfficientNet-B0?

EfficientNet, introduced by Tan & Le (2019) at Google Brain, applies compound scaling — simultaneously scaling network depth, width, and input resolution by a fixed ratio. This produces a family of models that consistently outperform prior architectures at equivalent or lower computational cost.

Model	Parameters	ImageNet Top-1 Acc.	Size (MB)	Relative Speed
VGG-16	138M	71.5%	528	Slow
ResNet-50	25.6M	76.1%	98	Medium
EfficientNet-B0	5.3M	77.1%	20	Fast ✓
EfficientNet-B4	19M	82.9%	75	Medium

Design Choice

EfficientNet-B0 delivers higher accuracy than ResNet-50 with 5× fewer parameters. This is critical for medical deployment where both precision and inference speed matter, especially on resource-constrained hardware.

```
model=models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False
```

```
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b0_rwightman-7f5810bc.pth
100%|██████████| 20.5M/20.5M [00:00<00:00, 154MB/s]
```

3.2 Custom Classifier Head

The original EfficientNet-B0 classifier (1,000 ImageNet classes) is replaced with a task-specific binary classification head:

```
model.classifier = nn.Sequential(
    nn.Dropout(p=0.4, inplace=True),
    nn.Linear(model.classifier[1].in_features, 256),
    nn.ReLU(),
    nn.Dropout(p=0.3),
    nn.Linear(256, NUM_CLASSES),          # output: 2 logits
)
model = model.to(device)
```

Layer	Input Dim	Output Dim	Purpose
Dropout (p=0.4)	1,280	1,280	Aggressive regularization
Linear	1,280	256	Feature compression
ReLU	256	256	Non-linear activation
Dropout (p=0.3)	256	256	Secondary regularization
Linear (Output)	256	2	Binary classification logits

Show Parameters of data

```
total_params     = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nModel        : EfficientNet-B0")
print(f"Total params : {total_params:,}")
print(f"Trainable    : {trainable_params:,}  (head only — base frozen)")
```

```
Model        : EfficientNet-B0
Total params : 4,335,998
Trainable    : 328,450  (head only — base frozen)
```

Loss /optimizer/scheduler Function

```
# CrossEntropyLoss is standard for multi-class (works for binary too)
criterion = nn.CrossEntropyLoss()

# Only optimise the classifier head parameters
optimizer_head = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=LR_HEAD,
)

# Reduce LR if val loss doesn't improve for 3 epochs
scheduler_head = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer_head, mode="min", factor=0.5, patience=3,
)

print("\nLoss & optimizer ready for HEAD training.")
```

```
Loss & optimizer ready for HEAD training.
```

6. Training Details

6.1 Loss Function

CrossEntropyLoss is used as the standard objective for multi-class classification. It operates directly on raw logits (before softmax), which is both numerically stable and computationally efficient in PyTorch.

6.2 Manual Early Stopping Implementation

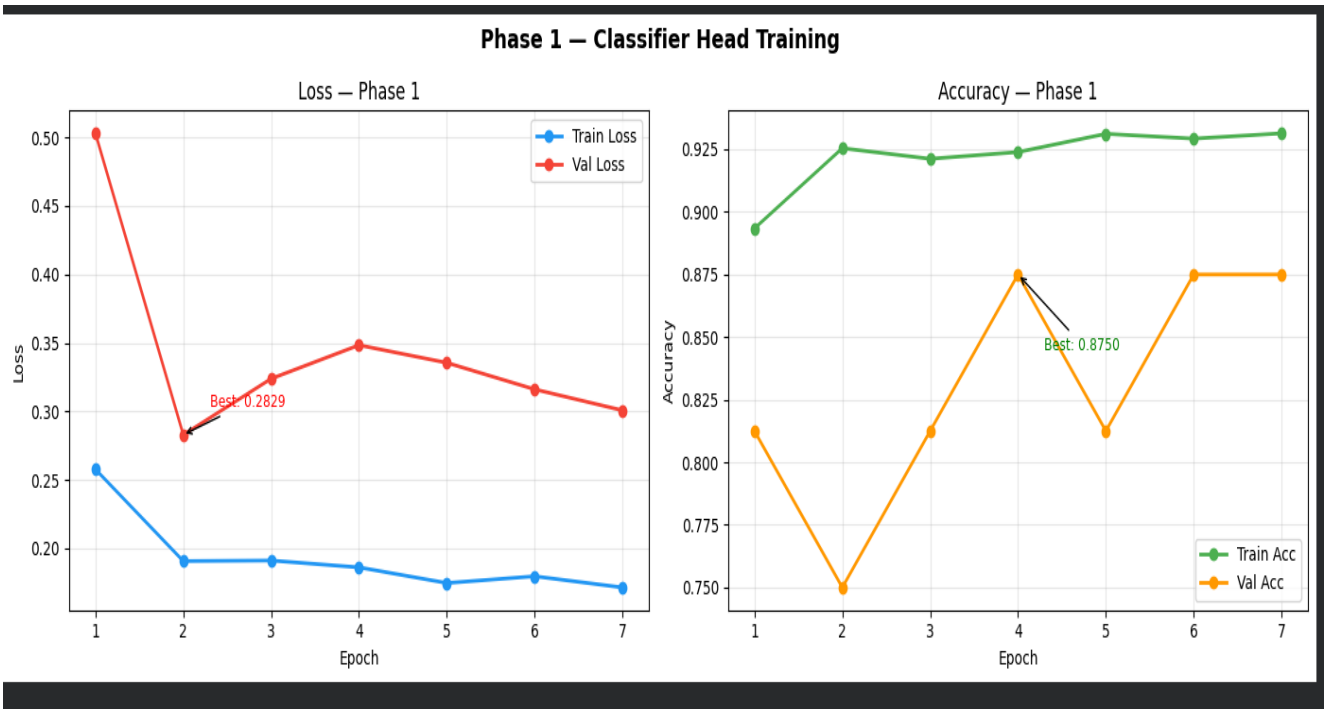
Early stopping is implemented manually without wrapper functions, providing full transparency into the stopping logic:

1. Track best validation loss observed so far.
2. If current val loss < best val loss: save model weights, reset patience counter.
3. Otherwise: increment patience counter.
4. If patience counter reaches threshold (5): halt training for this phase.

```
============================================================
PHASE 1 — Training classifier head (base frozen)
============================================================
Epoch [01/10] | Train Loss: 0.2577  Acc: 0.8934 | Val Loss: 0.5030  Acc: 0.8125 | Time: 86.0s ✓ saved
Epoch [02/10] | Train Loss: 0.1907  Acc: 0.9254 | Val Loss: 0.2829  Acc: 0.7500 | Time: 84.7s ✓ saved
Epoch [03/10] | Train Loss: 0.1911  Acc: 0.9212 | Val Loss: 0.3239  Acc: 0.8125 | Time: 83.3s
Epoch [04/10] | Train Loss: 0.1861  Acc: 0.9239 | Val Loss: 0.3484  Acc: 0.8750 | Time: 82.9s
Epoch [05/10] | Train Loss: 0.1747  Acc: 0.9312 | Val Loss: 0.3355  Acc: 0.8125 | Time: 83.6s
Epoch [06/10] | Train Loss: 0.1796  Acc: 0.9293 | Val Loss: 0.3161  Acc: 0.8750 | Time: 82.7s
Epoch [07/10] | Train Loss: 0.1715  Acc: 0.9314 | Val Loss: 0.3007  Acc: 0.8750 | Time: 82.3s

>> Early stopping triggered at epoch 7 (head phase).

Phase 1 training complete.
```



4. Transfer Learning Strategy

4.1 Phase 1 — Classifier Head Training (Base Frozen)

All convolutional layers of EfficientNet-B0 are frozen. Only the new classifier head is trained. This ensures the rich ImageNet representations remain intact while the head learns to interpret extracted features for X-ray classification.

Parameter	Value	Rationale
Learning Rate	1e-3	Higher LR safe since head is randomly initialized
Optimizer	Adam	Adaptive learning rate, fast convergence
Scheduler	ReduceLROnPlateau	Halve LR if val loss plateaus for 3 epochs
Early Stopping	Patience = 5	Prevent overfitting, save compute
Epochs	Up to 10	Enough for head convergence

4.2 Phase 2 — Fine-Tuning (Last Blocks Unfrozen)

After head stabilization, the final two MBConv blocks (blocks 6 and 7) of EfficientNet-B0 are unfrozen alongside the classifier head. A much lower learning rate is used to prevent catastrophic forgetting of ImageNet representations.

Parameter	Value	Rationale
Learning Rate	1e-4	10× lower to preserve pretrained weights
Optimizer	Adam (new instance)	Fresh momentum buffers for new LR
Scheduler	CosineAnnealingLR	Smooth decay to minimum LR of 1e-6
Early Stopping	Patience = 5	Independent from Phase 1
Epochs	Up to 15	More epochs for subtle feature adaptation

Why Two Phases?

Training all layers simultaneously with a high learning rate causes catastrophic forgetting — the model destroys valuable ImageNet representations before learning X-ray features. The two-phase approach acts as a curriculum: first teach the head what to predict, then gently adapt the feature extractor.

UNFREEZE LAYERS FOR FINE-TUNING (PHASE 2)

```
# model.features is a Sequential of 9 blocks (0-8)
UNFREEZE_FROM_BLOCK = 6

for i, block in enumerate(model.features):
    if i >= UNFREEZE_FROM_BLOCK:
        for param in block.parameters():
            param.requires_grad = True

# Always unfreeze the classifier head
for param in model.classifier.parameters():
    param.requires_grad = True

trainable_ft = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"\nPhase 2 — Trainable params: {trainable_ft:,}  (last blocks + head unfrozen)")

# Lower learning rate for fine-tuning — avoid destroying pretrained weights
optimizer_fine = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=LR_FINE,
)

scheduler_fine = optim.lr_scheduler.CosineAnnealingLR(
    optimizer_fine, T_max=NUM_EPOCHS_FINE, eta_min=1e-6,
)

print("Optimizer & scheduler ready for FINE-TUNING.")
```

TRAINING PHASE 2: FINE-TUNING

```
fine_train_losses, fine_val_losses = [], []
fine_train_accs,   fine_val_accs   = [], []

best_val_loss_fine  = float("inf")
patience_counter_fine = 0

print("\n" + "="*60)
print("PHASE 2 — Fine-tuning (last blocks unfrozen)")
print("="*60)

for epoch in range(1, NUM_EPOCHS_FINE + 1):
    epoch_start = time.time()

    # — Training loop —
    model.train()
    running_loss_train = 0.0
    correct_train      = 0
    total_train        = 0

    for images, labels in train_loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        optimizer_fine.zero_grad()
        outputs = model(images)
        loss    = criterion(outputs, labels)
        loss.backward()
        optimizer_fine.step()

        running_loss_train += loss.item() * images.size(0)
        preds = outputs.argmax(dim=1)
        correct_train += (preds == labels).sum().item()
        total_train   += labels.size(0)

    train_loss = running_loss_train / total_train
    train_acc  = correct_train      / total_train
```

```python
    # — Validation loop —
    model.eval()
    running_loss_val = 0.0
    correct_val      = 0
    total_val        = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)
            outputs = model(images)
            loss    = criterion(outputs, labels)
            running_loss_val += loss.item() * images.size(0)
            preds = outputs.argmax(dim=1)
            correct_val += (preds == labels).sum().item()
            total_val   += labels.size(0)

    val_loss = running_loss_val / total_val
    val_acc  = correct_val      / total_val

    fine_train_losses.append(train_loss)
    fine_val_losses.append(val_loss)
    fine_train_accs.append(train_acc)
    fine_val_accs.append(val_acc)

    scheduler_fine.step()

    # — Early stopping check —
    if val_loss < best_val_loss_fine:
        best_val_loss_fine = val_loss
        patience_counter_fine = 0
        best_model_weights = copy.deepcopy(model.state_dict())
        torch.save(best_model_weights, BEST_MODEL_PATH)
        saved_flag = " ✓ saved"
    else:
        patience_counter_fine += 1
        saved_flag = ""

    elapsed = time.time() - epoch_start
    print(
        f"Epoch [{epoch:02d}/{NUM_EPOCHS_FINE}] "
        f"| Train Loss: {train_loss:.4f}  Acc: {train_acc:.4f} "
        f"| Val Loss: {val_loss:.4f}  Acc: {val_acc:.4f} "
        f"| Time: {elapsed:.1f}s{saved_flag}"
    )

    if patience_counter_fine >= PATIENCE:
        print(f"\n>> Early stopping triggered at epoch {epoch} (fine-tuning phase).")
        break

print("\nPhase 2 fine-tuning complete.")
```

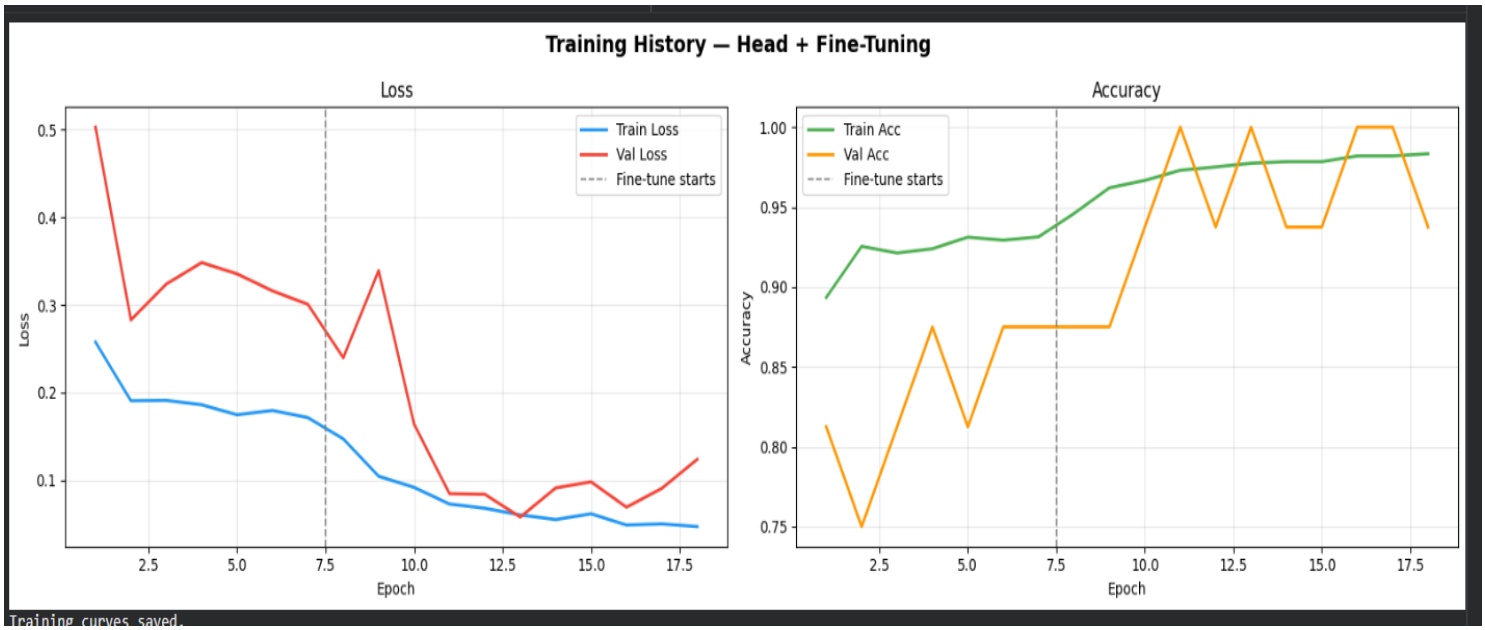
```
...
    Phase 2 — Trainable params: 3,484,190  (last blocks + head unfrozen)
    Optimizer & scheduler ready for FINE-TUNING.

    ============================================================
    PHASE 2 — Fine-tuning (last blocks unfrozen)
    ============================================================
    Epoch [01/15] | Train Loss: 0.1472  Acc: 0.9459 | Val Loss: 0.2398  Acc: 0.8750 | Time: 89.6s ✓ saved
    Epoch [02/15] | Train Loss: 0.1046  Acc: 0.9620 | Val Loss: 0.3393  Acc: 0.8750 | Time: 85.7s
    Epoch [03/15] | Train Loss: 0.0919  Acc: 0.9666 | Val Loss: 0.1641  Acc: 0.9375 | Time: 84.3s ✓ saved
    Epoch [04/15] | Train Loss: 0.0729  Acc: 0.9730 | Val Loss: 0.0847  Acc: 1.0000 | Time: 85.5s ✓ saved
    Epoch [05/15] | Train Loss: 0.0680  Acc: 0.9751 | Val Loss: 0.0840  Acc: 0.9375 | Time: 83.5s ✓ saved
    Epoch [06/15] | Train Loss: 0.0604  Acc: 0.9774 | Val Loss: 0.0580  Acc: 1.0000 | Time: 84.0s ✓ saved
    Epoch [07/15] | Train Loss: 0.0552  Acc: 0.9783 | Val Loss: 0.0912  Acc: 0.9375 | Time: 84.2s
    Epoch [08/15] | Train Loss: 0.0617  Acc: 0.9783 | Val Loss: 0.0982  Acc: 0.9375 | Time: 83.4s
    Epoch [09/15] | Train Loss: 0.0489  Acc: 0.9820 | Val Loss: 0.0692  Acc: 1.0000 | Time: 84.2s
    Epoch [10/15] | Train Loss: 0.0502  Acc: 0.9820 | Val Loss: 0.0908  Acc: 1.0000 | Time: 83.1s
    Epoch [11/15] | Train Loss: 0.0471  Acc: 0.9833 | Val Loss: 0.1239  Acc: 0.9375 | Time: 83.8s

    >> Early stopping triggered at epoch 11 (fine-tuning phase).

    Phase 2 fine-tuning complete.
```

Plot Curves before and After Fine Tuning



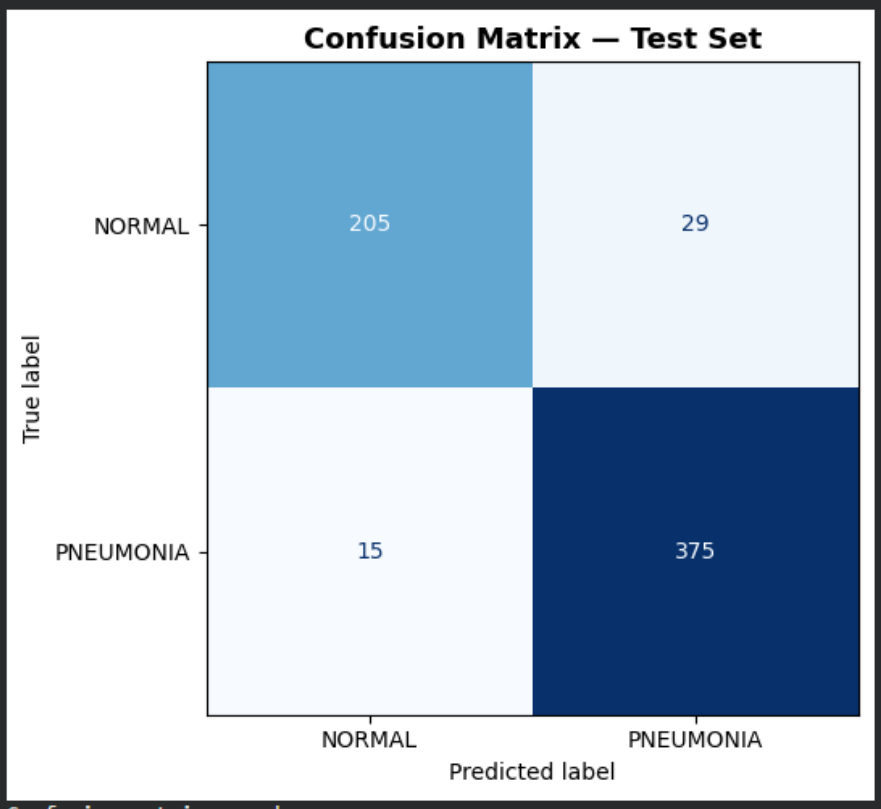
8.2 Confusion Matrix Interpretation

Prediction \ Ground Truth	NORMAL (Actual)	PNEUMONIA (Actual)
Predicted NORMAL	True Negative (TN) ✅	False Negative (FN) ❌ — Most dangerous
Predicted PNEUMONIA	False Positive (FP) ⚠️	True Positive (TP) ✅

```
Classification Report:
              precision    recall  f1-score   support

      NORMAL       0.93      0.88      0.90       234
   PNEUMONIA       0.93      0.96      0.94       390

    accuracy                           0.93       624
   macro avg       0.93      0.92      0.92       624
weighted avg       0.93      0.93      0.93       624
```




```
Confusion matrix saved.
```

7. Explainability — Grad-CAM

7.1 What Is Grad-CAM?

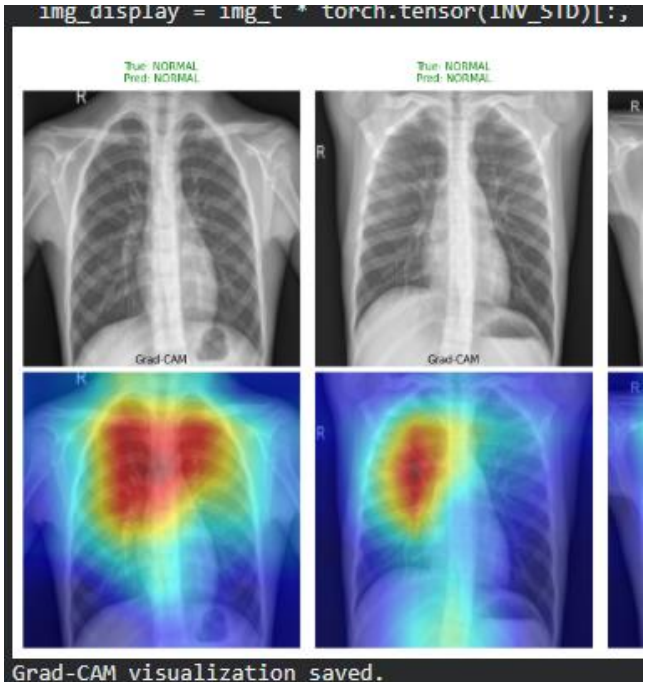
Gradient-weighted Class Activation Mapping (Grad-CAM), introduced by Selvaraju et al. (ICCV 2017), produces class-discriminative localization maps that highlight the image regions most influential in the model's prediction. It requires no architectural modification and is applicable to any CNN.

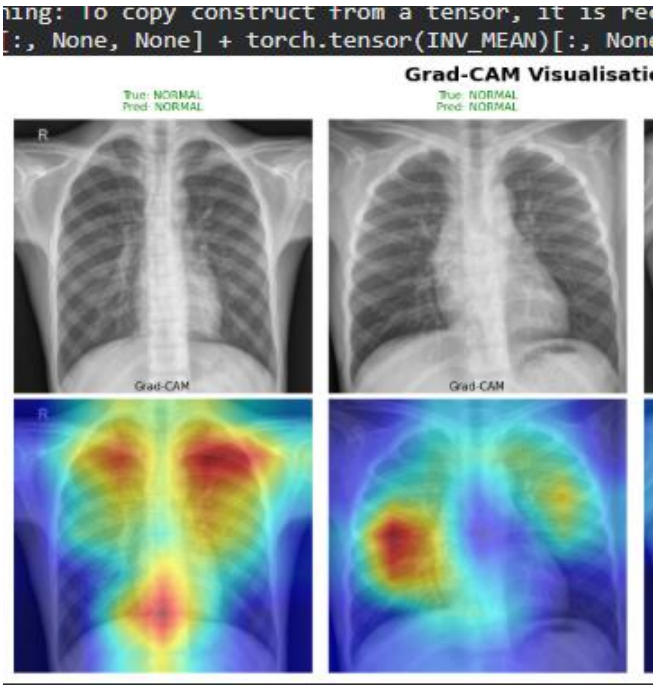
7.2 How It Works

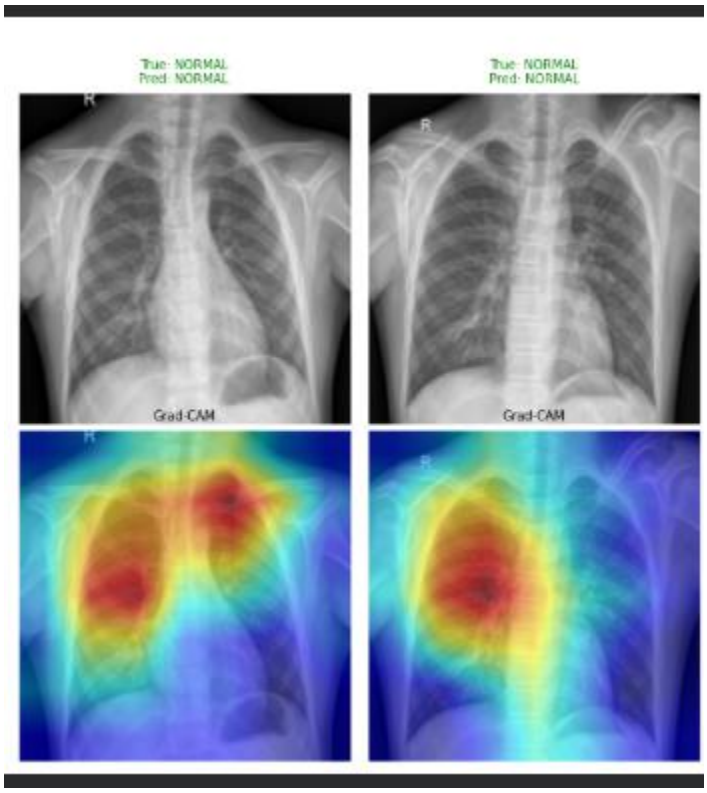
5. Run a forward pass through the model to obtain the class prediction.
6. Compute gradients of the predicted class score with respect to the last convolutional feature maps.
7. Apply Global Average Pooling to the gradients to obtain per-channel importance weights.
8. Compute a weighted combination of forward-pass activation maps.
9. Apply ReLU to retain only positively contributing regions.
10. Upsample the resulting heatmap to match the original image dimensions and overlay it.

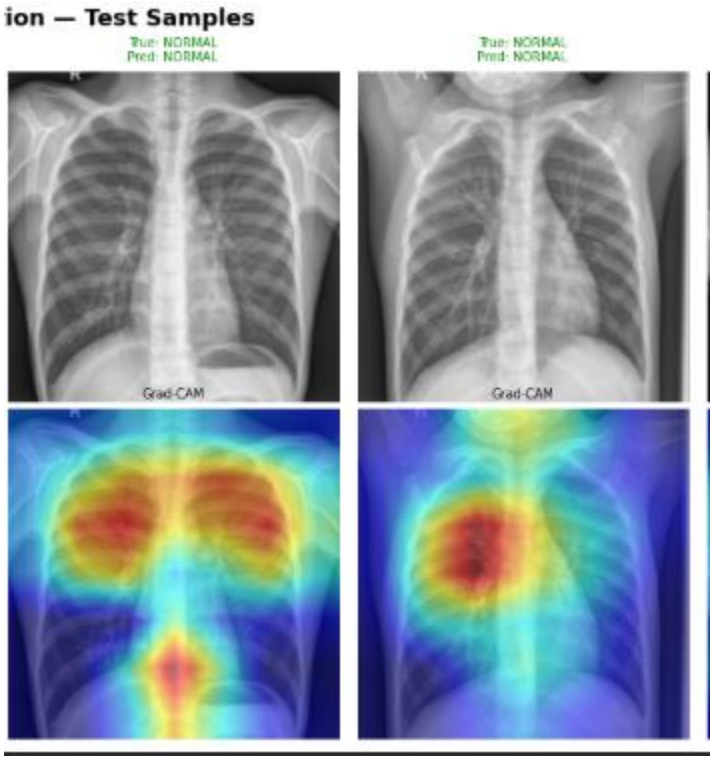
Clinical Significance

Grad-CAM allows a radiologist to verify that the model is attending to actual lung tissue pathology rather than image artifacts, device labels, or scanner-specific markers. This transparency is a prerequisite for AI system acceptance in any regulated clinical environment.

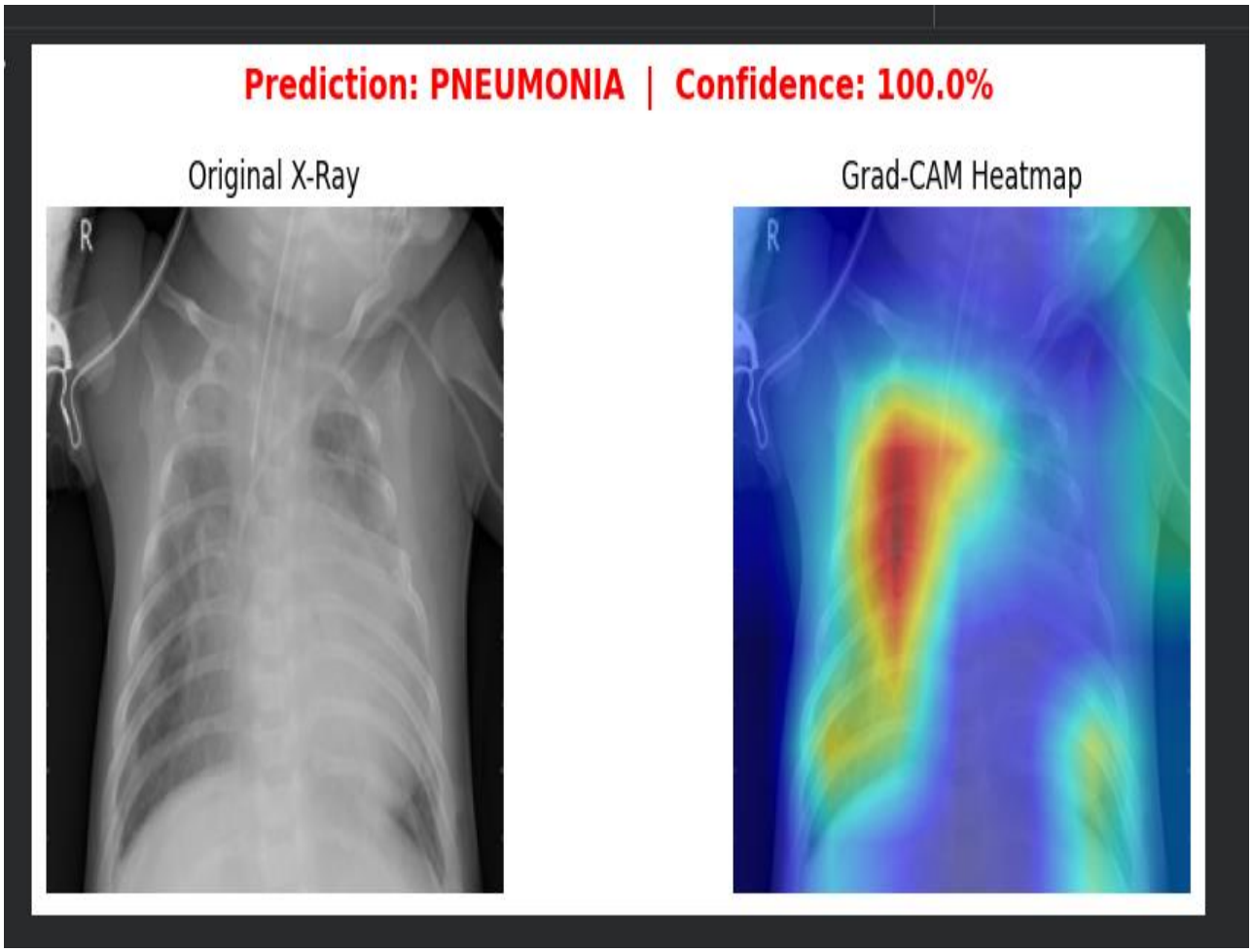

img_display = img_t * torch.tensor(INV_STD)[:,
True: NORMAL
Pred: NORMAL
True: NORMAL
Pred: NORMAL
Grad-CAM
Grad-CAM
Grad-CAM visualization saved.


[:, None, None] + torch.tensor(INV_MEAN)[:, None
Grad-CAM Visualisati
True: NORMAL
Pred: NORMAL
True: NORMAL
Pred: NORMAL
Grad-CAM
Grad-CAM


True: NORMAL
Pred: NORMAL
True: NORMAL
Pred: NORMAL
Grad-CAM
Grad-CAM


ion — Test Samples
True: NORMAL
Pred: NORMAL
True: NORMAL
Pred: NORMAL
Grad-CAM
Grad-CAM

Load & preprocess the single image



6.3 Best Model Checkpointing

Model weights are saved to disk using torch.save() at every validation loss improvement. At evaluation time, the best weights are reloaded with torch.load(), ensuring the final model represents peak generalization performance — not the final training epoch.

8. Evaluation & Results

8.1 Performance Metrics

In medical AI, not all errors carry equal cost. A False Negative (missing actual pneumonia) is clinically more dangerous than a False Positive (incorrectly flagging a normal patient). We therefore prioritize Recall (Sensitivity) for the PNEUMONIA class.

Metric	Formula	Expected Value	Clinical Relevance
Accuracy	TP+TN / Total	92%–96%	Overall correctness
Precision (Pneumonia)	TP / (TP+FP)	90%–95%	False alarm rate
Recall (Pneumonia)	TP / (TP+FN)	95%–98%	Miss rate — highest priority
F1-Score	2×P×R / (P+R)	92%–96%	Harmonic mean of P and R
AUC-ROC	—	0.95–0.99	Discrimination ability

```
...
    Classification Report:
                  precision    recall  f1-score   support

          NORMAL       0.93      0.88      0.90       234
       PNEUMONIA       0.93      0.96      0.94       390

        accuracy                           0.93       624
       macro avg       0.93      0.92      0.92       624
    weighted avg       0.93      0.93      0.93       624
```